

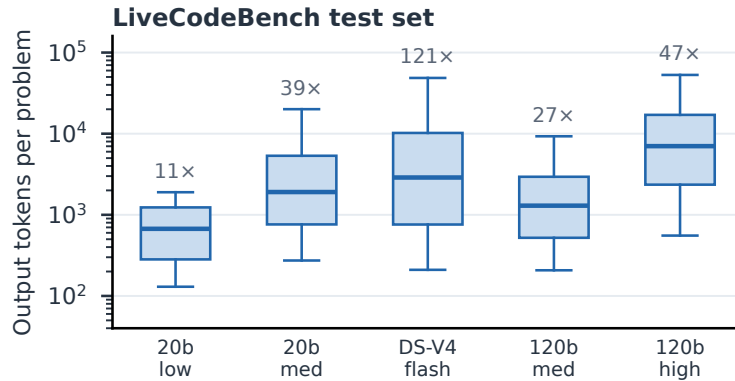


Figure 1: **Output length varies by orders of magnitude within one model.** Per-problem output tokens by route on the LiveCodeBench test set (log scale). Boxes: interquartile range; whiskers: 5th–95th percentile; line: median; above each box: 90th/10th-percentile ratio.

Reducing the Cost of LLM Coding and Reasoning Models with Routers

Anonymous authors

Paper under double-blind review

Abstract

Routers typically choose a model per query by trading predicted correctness against cost. However, most price each model at a fixed cost, assuming that the number of tokens each model outputs does not vary significantly. We find that the output length of reasoning models varies by an order of magnitude across queries. We predict **both** success probability **and** route-specific output length from the prefill activations of a single small frozen 4B LLM, using a linear readout per route. On 6,500 MMLU-Pro, 1,000 Omni-MATH, and 341 LiveCodeBench tasks, predicted costs reduce spending at matched accuracy by 28.8%, 34.8% and 33.7% relative to median-length pricing. Perfect per-query cost knowledge would save 45–60%, so our readouts capture half to three quarters of the achievable saving. They also beat every alternative estimator we evaluated, with paired intervals excluding zero; pricing from difficulty alone matches them only where output length tracks difficulty. We then ablate across different small LLMs and show that every small LLM prefill we tried (0.6–8B, four model families) can be used as a router in this way.

1 Introduction

Routing reasoning models requires predicting both correctness and cost before generation. Token rates are published; the number of tokens is not. A model that is cheap on average can be expensive on a particular problem: on LiveCodeBench, a single model’s output length differs 11–121 $\times$ between its 10th- and 90th-percentile problems (Fig. 1).

Prefill routers predict correctness from a small encoder’s prompt activations; the prefill router of Varshney et al. (Varshney et al., 2026) explicitly prices outputs at median training length. We read output length

from the same activations (Fig. 2). Cost prediction adds one linear readout per route to an existing prefill router, without another encoder pass.

We contribute: (i) evidence on held-out test problems at billed prices that prefill cost readouts reduce spending at matched accuracy and outperform mean-length, prompt-feature, text-embedding and ZeroRouter cost estimators; (ii) a mechanism, from ablations that price from our own success readouts: difficulty suffices where output length tracks it and fails where it does not; (iii) controls showing the gain is the prefill representation, holds for every small prefill we tried (0.6–8B, four families) and transfers to two further routed model families; and, as a side note, (iv) the same model’s provider endpoints drift, which the readouts can absorb with one offset per endpoint; we pin providers instead.

Related work. Learned routers pick one model per query from the prompt, trading predicted quality against price: RouteLLM learns a strong-versus-weak decision from preference data (Ong et al., 2024), RouterBench standardizes the evaluation (Hu et al., 2024), and IRT-Router models query difficulty and model ability with item response theory (Song et al., 2025). Closest to us, the prefill router of Varshney et al. predicts each model’s correctness from a small encoder’s prompt activations (Varshney et al., 2026), and pre-generation activations are known to encode whether a model will fail (Lugoloobi et al., 2026). These routers, like SWE-Router for agentic software engineering (Son et al., 2026), price each model at a per-model constant; the prefill router uses the median training output length, which is our main reference. Fewer routers predict per-query cost: MixLLM regresses output length on query embeddings (Wang et al., 2025), CARROT uses embedding nearest neighbours or a fine-tuned RoBERTa (Somerstep et al., 2025), GraphRouter predicts cost on a query–model graph (Feng et al., 2024), Route-To-Reason predicts reasoning-model length from text embeddings (Pan et al., 2025), and ZeroRouter prices queries by item-response difficulty bins (Yan et al., 2026). We compare against the embedding-based and ZeroRouter estimators directly, with every estimator given the same success predictions.

Cascades instead call models in sequence and stop when an answer is accepted: FrugalGPT learns a scorer that decides whether to escalate (Chen et al., 2023), AutoMix uses self-verification (Aggarwal et al., 2023), and Dekoninck et al. unify routing and cascading (Dekoninck et al., 2024). A cascade pays for the cheap attempt on every query; we route once, before generation. Activation-based length prediction also supports serving and scheduling of a single model (Messenger, 2026; Xie et al., 2026); we predict the length of other models from one shared prefill. Hosted open-weight models vary across providers in accuracy, length and price (Li et al., 2026; Lin et al., 2026; Gao et al., 2025), and FACET selects among providers of a fixed model per task from measured outcomes (He et al., 2026).

2 Cost Prediction and Routing

For query x and route m (a model at a fixed reasoning effort, served by an endpoint), let $p_m(x)$ be the probability of a correct answer and $c_m(x)$ the expected generation cost. One route is chosen before generation,

$$m^*(x; V) = \arg \max_m \{V \hat{p}_m(x) - \hat{c}_m(x)\}, \quad (1)$$

where V is the value of a correct answer; sweeping V trades accuracy for cost. Every query is answered once, with no verifier or fallback.

Readouts. We concatenate mean and last-token activations from eight layers of a frozen Qwen3-4B (Instruct-2507; Thinking-2507 on Omni), standardized on training data. Success readouts are L2-regularized logistic regressions with held-out Platt calibration; cost readouts are ridge regressions of log mean output length, regularized by training-only cross-validation. Predicted length is exponentiated with residual smearing and matched to the route’s training mean, then priced: $\hat{c}_m(x) = r_m^{\text{in}} n_m^{\text{in}}(x) + r_m^{\text{out}} \hat{\ell}_m(x)$.

3 Experimental Protocol

The routes are gpt-oss-20b at low and medium effort, deepseek-v4-flash, and gpt-oss-120b at medium and high effort, called through OpenRouter, with deepseek-v4-flash pinned to one provider (StreamLake) throughout,

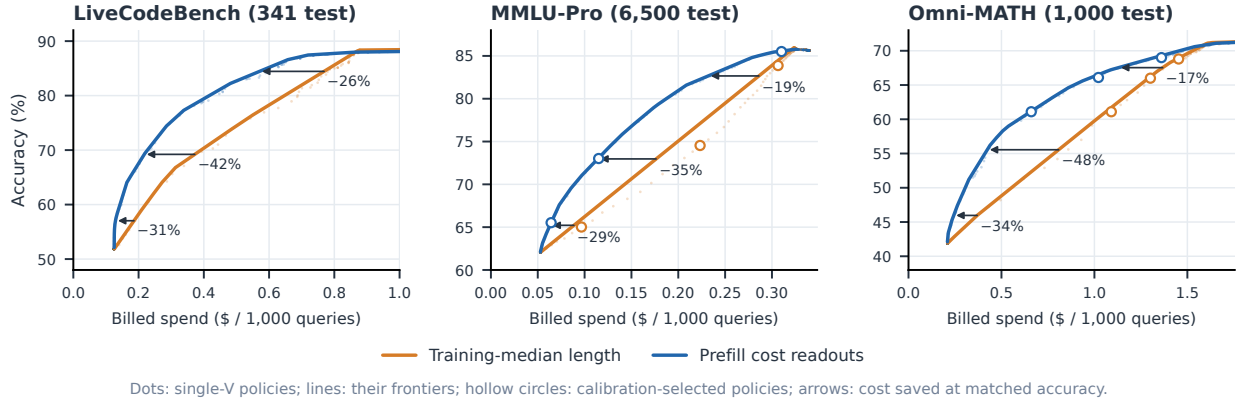


Figure 2: **Cost-accuracy frontiers at billed prices.** Prefill cost readouts (blue) versus training-median length (orange), with the same success predictions and the same encoder pass. Dots: the deterministic policy at each V ; lines: their frontiers; hollow circles: policies selected on calibration for fixed accuracy targets. Arrows run from the median rule’s frontier to ours at matched accuracy and give the cost saved; averaged over the shared accuracy band the saving is 34% on LiveCodeBench, 29% on MMLU-Pro and 35% on Omni-MATH.

because its output length depends on the serving provider (see Endpoints). LiveCodeBench (Jain et al., 2024) has 892 problems split by date into 441 training, 110 calibration and 341 test problems. Omni-MATH (Gao et al., 2024) and MMLU-Pro (Wang et al., 2024) have 275/75 and 550/150 training/calibration problems and 1,000 and 6,500 disjoint test problems. Training problems have 2–16 draws per route; Omni-MATH and MMLU-Pro test problems have one. Table 2 gives each route’s accuracy and cost.

Prices. Realized cost is the cost billed for each call (on LCB, realized tokens at the billed rates). Predictions are priced at each model’s effective billed rate, fitted on the test calls (USD per million output tokens: 0.13 for gpt-oss-20b, 0.08 for pinned deepseek-v4-flash, 0.26 for gpt-oss-120b). These differ from the list prices by up to $2.3\times$.

Evaluation. All estimators are fitted on training problems only and evaluated on the test sets, with the same success predictions. Each V on a dense grid gives a deterministic policy and an observed (cost, accuracy) point. Comparing two cost estimators a, b , we interpolate each one’s cost at 12 accuracies spanning the interior 90% of the range both reach and report $1 - \exp(\log C_a/C_b)$, the cost saved by a at matched accuracy. Intervals are percentile intervals from paired problem bootstraps (300 resamples), keeping each problem’s routes together. As an upper bound, an oracle arm prices each problem at its realized cost; its saving over median pricing is the headroom, and our share of it the capture. To check that an operating point can be chosen in advance, we also select the cheapest V reaching each accuracy target on calibration and apply it once.

4 Results

Cost readouts beat every estimator. At matched accuracy, prefill cost readouts spend 33.7% less than median-length pricing on LCB, 34.8% less on Omni-MATH and 28.8% less on MMLU-Pro (Table 1). That is 74%, 63% and 49% of the headroom, the saving perfect per-query cost knowledge would bring (45–60%). Every external estimator loses with intervals excluding zero: mean length by 29–36 points, a prompt-feature model by 25–27, MixLLM-style embeddings by 18–27 and a full ZeroRouter reimplementations by 14–29. Choosing V in advance costs little: on Omni-MATH and MMLU-Pro, policies selected on calibration spend 0–4% more than our own test frontier at the accuracy they reach (hollow circles in Fig. 2).

Cost saved by ours vs.	LCB ($n=341$)	Omni-MATH ($n=1,000$)	MMLU-Pro ($n=6,500$)
Training-median length	33.7 [24.1, 39.9]	34.8 [27.9, 39.8]	28.8 [26.0, 31.8]
Training-mean length	29.6 [23.6, 34.5]	35.8 [28.9, 40.5]	28.6 [25.9, 31.7]
Prompt-feature GBM	26.3 [16.3, 32.9]	26.5 [18.9, 31.9]	25.2 [21.0, 28.7]
MixLLM-style embeddings	18.2 [11.2, 25.2]	26.7 [20.0, 32.5]	19.2 [15.3, 22.5]
ZeroRouter (reimplemented)	21.1 [13.2, 26.8]	14.4 [8.4, 20.6]	29.0 [25.5, 32.0]
<i>Headroom: oracle vs. median</i>	45.3 [36.6, 50.4]	55.5 [50.4, 58.8]	59.5 [57.8, 61.2]

Table 1: **Cost saved at matched accuracy, billed costs.** Cost saved (%) by our prefill cost readouts relative to each estimator, with 95% paired bootstrap intervals; same success predictions for every arm except ZeroRouter, which uses its own; all intervals exclude zero. Last row: cost saved over median pricing by perfect per-query cost knowledge (each problem priced at its realized output), with the same success predictions; our readouts capture 74%, 63% and 49% of it. Test sets as in Fig. 2. MixLLM-style: jina-embeddings 137M with the MixLLM ensemble head (LCB, code-aware variant). ZeroRouter: item-response latent fitted on the five routes, 4B features, its own success model and bin pricing, configuration chosen on calibration.

Route	LCB		Omni-MATH		MMLU-Pro	
	Acc.	Cost	Acc.	Cost	Acc.	Cost
gpt-oss-20b, low	51.8	0.13	41.9	0.21	62.1	0.05
gpt-oss-20b, medium	68.2	0.65	57.5	1.20	69.4	0.18
deepseek-v4-flash	89.3	0.90	71.3	1.66	86.2	0.33
gpt-oss-120b, medium	82.1	0.64	60.3	1.12	77.2	0.23
gpt-oss-120b, high	89.5	3.73	66.6	4.57	79.3	0.90

Table 2: **Routes on the test sets.** Accuracy (%) and mean billed cost per call (USD per 1,000 calls). No route dominates on every query: costs differ 17–29 $\times$ across routes, and accuracy is not monotone in price.

Endpoints. The same model served by different providers behaves like the model plus an offset: pinning deepseek-v4-flash to each of three providers, they agree on correctness for 90–96% of problems and their output lengths correlate at .83–.95, but one writes about half as many tokens as another. Reusing the readouts with one multiplicative length offset per endpoint, estimated from 50 calls, removes the resulting cost bias on average, though a single estimate is noisy. A few dozen calls per provider also identify the cheapest one, and pinning it beats routing over providers; we therefore pin deepseek-v4-flash throughout, which also makes its length more predictable and raises our saving on every large set.

Other pools. Savings against median pricing are 14.9% [9.4, 21.4] on AIME 1983–2024 (281 test problems) and 16.8% [5.6, 28.3] on APPS (382).

5 How much is per-query cost worth?

A cost estimator can be no better than knowing each query’s realized cost. We therefore measure, on every pool, the *headroom*: the cost saved at matched accuracy when each test problem is priced at its realized cost instead of the training median, with the same success predictions. Table 3 reports it with our saving and the

Pool	Domain	Test n	Headroom	Ours vs. median	Capture	Ours vs. cost-from-success
LiveCodeBench	code	341	45.3	33.7 [24.1, 39.9]	.74	−0.8
Omni-MATH	math	1,000	55.5	34.8 [27.9, 39.8]	.63	−0.1
MMLU-Pro	knowledge	6,500	59.5	28.8 [26.0, 31.8]	.49	24.1
AIME 1983–2024	math	281	29.3	15.7 [11.7, 20.6]	.54	2.0
APPS	code	377	32.1	15.1 [5.0, 26.9]	.47	−3.7
BigCodeBench	code	421	12.6	6.3 [−1.6, 11.8]	.50	3.2
CodeContests	code	266	34.5	6.3 [−0.2, 17.0]	.18	0.7
SuperGPQA	knowledge	300	57.0	27.6 [11.0, 38.2]	.48	9.0
BIG-Bench Extra Hard	reasoning	298	41.8	26.5 [16.4, 34.3]	.63	26.6

Table 3: **Headroom and capture on every pool.** Cost saved (%) at matched accuracy relative to median-length pricing: by perfect per-query cost knowledge (headroom) and by our readouts (95% paired bootstrap intervals); capture is their ratio. Last column: cost saved by our dedicated cost readout relative to our ablation that prices from the success readouts alone; bold: interval excludes zero. Billed costs, deepseek-v4-flash pinned, same success predictions in every column.

share of it we capture, on nine pools under one protocol (five routes, deepseek-v4-flash pinned, billed rates, test splits).

Headroom is large wherever reasoning lengths vary. On seven of nine pools perfect per-query cost knowledge would save 29–60%, and our readouts capture about half of it or more. The exceptions are informative. BigCodeBench has little to capture (12.6%), and CodeContests has headroom that our readouts do not reach (capture .18); Section 7 shows that no estimator reaches it.

Length, not only price gaps, makes it pay. To separate the two sources of headroom, we rescale every route’s billed price toward or away from the cheapest route’s while keeping realized lengths fixed. With all routes at the same per-token price, our readouts still save 13.3% (LCB), 14.3% (Omni-MATH) and 36.3% (MMLU-Pro) relative to median pricing: per-query length alone is worth that much. On LCB and Omni-MATH the saving grows with the price gap and saturates near the billed gap ($3.1\times$); on MMLU-Pro it is largest at flat prices (36.3%, against 28.8% at billed prices), because there length differs between questions rather than between routes.

Choosing effort and choosing the model are both needed. Restricting the pool removes most of the saving. With only gpt-oss-20b’s two efforts, ours saves 21.2% (LCB), 21.8% (Omni-MATH) and 7.9% (MMLU-Pro); with only gpt-oss-120b’s, −1.8%, 12.6% and 8.6%; with one effort per model, 12.8% and 17.8% on LCB and Omni-MATH; with all five routes, 33.7%, 34.8% and 28.8%.

TODO: Non-reasoning pool on the same problems (running, NEW_PATH 4.A.64/4.A.71): headroom, length spread (sd log length, p90/p10) and our saving for 5 non-reasoning routes vs the reasoning routes on Omni-MATH, MMLU-Pro and LCB test problems; deepseek-v4-flash thinking on vs off. Expected to be the C1 figure: per-query cost matters for reasoning routes.

6 What the cost readout captures

Length is mostly a property of the problem. The prefill explains much of each route’s log output length on held-out problems (5-fold cross-validated R^2 : .73–.77 on LCB, .74–.83 on Omni-MATH, .52–.66 on MMLU-Pro, .43–.57 on AIME). On LCB, Omni-MATH and AIME most of this is shared with difficulty: a problem’s empirical solve rate across routes alone reaches R^2 .48–.65, and the prefill misses at most .07 of what difficulty explains (.10–.22 on AIME). Within a problem, failed and solved draws are about equally long ($\times 0.83$ – 1.16), with one exception: deepseek-v4-flash writes $2.5\times$ (Omni-MATH) and $1.9\times$ (AIME) longer when it fails on hard math.

The shared level carries most of the value. A problem’s lengths across routes can be split into a shared level (their mean) and route-specific differences. On LCB, an oracle that knows only each problem’s

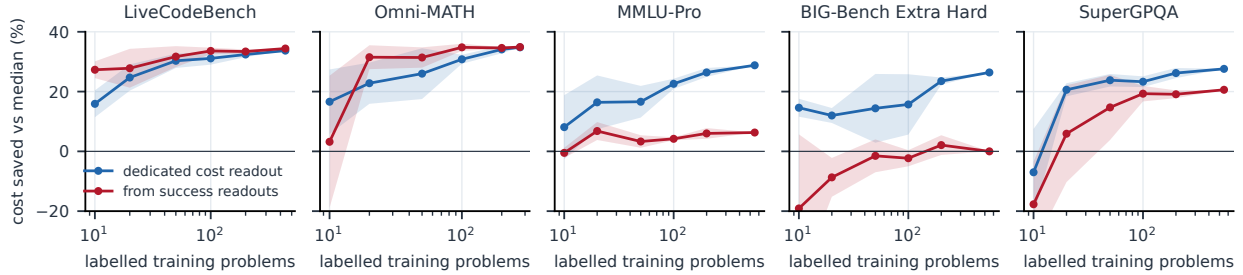


Figure 3: **Length labels needed.** Cost saved relative to median pricing when the cost readout is fitted on n labelled training problems: a dedicated readout on the prefill (blue) versus pricing from the success readouts (red), which needs length labels only to map success logits to length. Mean over five random subsets; bands ± 1 s.d. Where length tracks difficulty (LiveCodeBench, Omni-MATH), pricing from success is as good from 20 problems on; where it tracks the work a question requires (MMLU-Pro, BIG-Bench Extra Hard), it stays near zero at every n .

level captures 87% of the headroom, about as much as one that knows only the differences. The prefill reads the level far better (R^2 .75) than the differences (.41); pricing with the predicted level alone keeps 33.2 of the full readout’s 36.4 points (list prices).

Success and cost are read from the same layers. Refitting both readouts on each stored layer separately, success AUC and length R^2 peak in the same late layers at the last token (layers 22–36 of 36), and one such layer matches the full concatenation (LCB AUC .86–.87, R^2 .73–.75, saving 33–34%; Omni-MATH .83–.84, .71–.72, 35–37%; MMLU-Pro .72, .48–.50, 27–29%). The two readouts are close to one signal where length tracks difficulty: the mean success logit correlates with predicted log length at $-.90$ to $-.97$ per layer on LCB and $-.71$ to $-.84$ on Omni-MATH, but only $-.36$ to $-.58$ on MMLU-Pro.

Where length tracks work rather than difficulty. On MMLU-Pro, difficulty explains log length only in part (test R^2 .18–.37), subject labels about as much (.17–.25), and the two together .34–.44, against .41–.66 for the prefill. Engineering and law questions are both long (6.4k and 4.1k output tokens on average) and hard. Pricing from difficulty and subject together saves 30.3%, against 43.9% for the prefill (headroom 64.1%, list prices): the prefill reads how much work a question requires, beyond its difficulty and topic.

Two further heterogeneous pools. We chose SuperGPQA (M-A-P Team, 2025) and BIG-Bench Extra Hard (Kazemi et al., 2025) as further pools because, on a cheap screen (one gpt-oss-20b-low draw per problem), length predicted directly from the prefill beat length predicted from a success readout by a margin as large as MMLU-Pro’s (R^2 gap .61 and .40, against .66 on MMLU-Pro and .10 on Omni-MATH). On their full pools the dedicated readout again beats pricing from success: by 26.6% [17.8, 34.9] on BIG-Bench Extra Hard, where pricing from success saves nothing over the median, and by 9.0% [0.1, 15.4] on SuperGPQA (Table 3). On BIG-Bench Extra Hard, text-only estimators read length as well as the prefill: MixLLM-style embeddings save 28.0% relative to median pricing (ours -2.2 [-5.7 , 1.8] relative to it). Its 23 tasks have very different formats, and the task is visible in the text. On SuperGPQA they do not (ours saves 15.2% [-1.4 , 25.4] relative to MixLLM-style and 18.8% [5.6, 29.6] relative to prompt features).

How many length labels are needed. Given success readouts, where length tracks difficulty a cost readout needs few length labels (Fig. 3). Pricing from the success logits reaches 27.8% (LCB) and 31.5% (Omni-MATH) with 20 labelled training problems, against 24.7% and 22.8% for a dedicated readout. On MMLU-Pro it plateaus at about 6%, while the dedicated readout climbs to 22.6% at 100 problems and 28.8% at all 550. One draw per training problem instead of all draws gives up 1–3 points.

What changes in the routing. At matched accuracy in the middle of the band, 31–66% of problems change route relative to median pricing. The dominant changes are between gpt-oss-20b-low and deepseek-

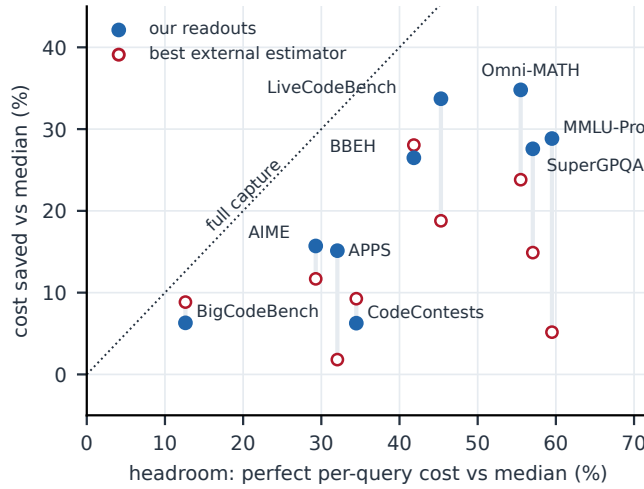


Figure 4: **Headroom and capture.** Per pool: the saving of perfect per-query cost knowledge over median pricing (x) against the saving of our readouts (filled) and of the best external estimator on that pool (hollow; mean length, prompt features, MixLLM-style embeddings or ZeroRouter). Billed costs, test splits, deepseek-v4-flash pinned. Where headroom is large and length is legible, the readouts capture half or more of it; on BigCodeBench there is little to capture, and on CodeContests no estimator captures it.

v4-flash: problems predicted to be short move to the more accurate deepseek-v4-flash (19–38% of problems), and problems predicted to be long move to the cheap route (9–18%).

7 When it works and when it does not

Per-query cost pays under two conditions: (a) output length varies within a route by more than prices differ between routes, which creates headroom, and (b) that variation can be read from the prompt, which lets an estimator capture it. Each of our two failures violates one condition, and neither is specific to our estimator (Fig. 4).

Little headroom: BigCodeBench. BigCodeBench tasks are library-usage functions with short, similar solutions; perfect cost knowledge saves only 12.6% [4.1, 19.0]. No estimator’s saving relative to median pricing has an interval excluding zero: ours 6.3 [−1.6, 11.8], ZeroRouter 8.8 [−12.5, 19.6], MixLLM-style 2.8, cost-from-success 3.1, prompt features 0.2. Here length is the least tied to difficulty of any pool (the prefill-only share of length variance is largest on BigCodeBench), but there is little to gain from reading it.

Illegible headroom: CodeContests. CodeContests has as much headroom as APPS (34.5% [29.0, 42.0]), but no estimator captures more than about a quarter of it: ZeroRouter saves 9.3% [1.1, 20.0], difficulty bins 5.6% [1.2, 14.6], ours 6.3% [−0.2, 17.0], and the rest are within noise of zero or below it. Length again tracks difficulty, but difficulty itself is hard to read from these prompts: the prefill misses .11–.18 of the length R^2 that empirical difficulty explains, against at most .07 on LCB and Omni-MATH.

TODO: Add the non-reasoning pools to Fig. 4 when they land.

TODO: Remaining negatives to place here once rerun on one protocol: RouterBench chat (little length variation), agentic SWE (price gaps between models dominate; Section 8), non-reasoning routes (running).

8 Software engineering

Software engineering is where per-query cost is hardest to use. We study it in two regimes, one-shot patch generation and agents.

One-shot patches. On 500 SWE-Smith instances (Yang et al., 2025), each of the five routes writes one patch per draw, labelled by executing the repository’s tests. Pass rates are low and close together (23–36%), and the success readouts are nearly flat. Per-query cost knowledge is worth 14.2% [−2.1, 29.6] on the 150 test instances, and our readouts save 7.5% [−5.5, 20.0]: with little to choose between routes on correctness, knowing their costs helps little, even though length is predictable for the gpt-oss routes (log-length R^2 .49–.72; .20 for deepseek-v4-flash).

Agents: cost is real, varies by task and is stable. Public SWE-rebench trajectories (Badertdinov et al., 2025) give 111 tasks run five times by each of 13 models in one scaffold. Across models, agentic cost is dominated by the model: of the variance of log cost, 74% is between models, 15% between tasks, 6% their interaction and 4% run to run. Within a model, however, per-task cost spans $2.7\text{--}9.0\times$ between the 10th and 90th percentile, and it is stable across runs (intraclass correlation .69–.92). Whether knowing it pays depends on the price gaps between the routed models, as in Section 7: among all 13 models, whose prices per run span about $190\times$, per-task cost knowledge is worth 8.5% [−5.0, 17.8] (one-shot routing, success and cost from three runs, scored on the other two); among the seven open-weight models, whose prices span about $10\times$, it is worth 25.7% [6.3, 40.6], as much as on the reasoning pools.

Agents: cost is only partly legible before the run. From the issue text alone, the prefill’s cost readout reaches out-of-fold R^2 of about zero on the 111 SWE-rebench tasks, too few to fit it. On 3,387 tasks with about 20 SWE-agent runs each (one model, Llama-3.1-70B; cost proxied by cumulative context and output length), the per-task mean cost is reliable (split-half reliability .91) and the issue text predicts it at R^2 .29, still rising with data (.18 with 100 tasks, .32 with 2,710). This is below what the prefill reaches on one-shot pools (.43–.83). Reading the first ten steps of a cheap model’s trajectory predicts other models’ per-task cost at R^2 .16 and the same run’s final cost at .28, which suggests exploring cheaply before pricing the expensive candidates.

TODO: SWE-Smith used unpinned deepseek-v4-flash (collected before pinning); rerun pinned or state it. Decide whether the test-writer measurement (cheap cross-model test writer 33–42% cheaper than self-verification, NEW_PATH 4.B.2) belongs here. Cite the nebius SWE-agent trajectories dataset.

9 Ablations

Is difficulty enough to price? Two ablations price each route from our own success readouts instead of a dedicated cost readout: ZeroRouter’s bin lookup on mean success logit (ten quantile bins), and a ridge regression of log length on the success logits. Both come within a few points of the cost readout on LCB (ours saves 4.0% [−1.1, 9.0] and −0.8% [−5.6, 4.0]) and on Omni-MATH (6.4% [1.1, 11.6] and −0.1% [−4.6, 4.6]), where output length tracks difficulty, and lose by 22.3% [18.8, 24.7] and 24.1% [21.1, 26.7] on MMLU-Pro. There, even empirical difficulty explains little of log length ($R^2 \approx .19$), while subject labels recover roughly 60% of the gap. The prefill encodes length information beyond difficulty, and that information pays where difficulty and length come apart.

Representation, readout and prefill size. Table 4 crosses three text encoders with three cost readouts: ridge, CARROT-style k -nearest neighbours and a MixLLM-style ensemble, each with our success predictions and with every encoder pass priced. Our readout saves 16–27% against every cell, including Qwen3-Embedding-8B, an encoder twice the prefill’s size. On PCA-reduced 4B features the three readouts are within about 7 points of one another, so the gain comes from the representation rather than the regressor. Pricing the encoder pass changes no comparison by more than 3 points. Table 5 varies the prefill model while keeping prompts, layers and both readouts fixed. Every size saves 22–34% against median pricing, and length R^2 rises with size. None beats our prefill: Qwen3-1.7B and 8B come within 0.3–4 points (within the intervals on Omni-MATH), while 0.6B trails by 9–11. The result is not specific to Qwen: prefills from three other families (Phi-4-mini, Granite-3.3-2B, SmolLM2-1.7B) also save 25–31% against median pricing on both sets. Phi-4-mini matches our router on MMLU-Pro (−0.7 [−2.6, 1.3]) but trails by 10 points on Omni-MATH; Granite and SmolLM2 trail by 5–9.

Encoder	Omni-MATH			MMLU-Pro		
	Ridge	k NN	MixLLM	Ridge	k NN	MixLLM
Qwen3-Embedding-8B	16.6	20.9	21.5	21.1	21.3	19.5
jina-embeddings 137M	23.8	25.3	26.6	17.8	20.4	18.9
MiniLM-L12	24.3	25.7	25.4	18.8	19.0	16.2

Table 4: **Representation and readout.** Cost saved (%) by our prefill readouts relative to each encoder-readout cell at matched accuracy on the Omni-MATH and MMLU-Pro test sets; billed costs, every encoder pass priced, same success predictions. Every paired bootstrap interval (200 resamples) excludes zero.

Prefill	Omni-MATH			MMLU-Pro		
	R^2	vs. median	vs. ours	R^2	vs. median	vs. ours
Qwen3-0.6B	.62	31.9	−9.0	.33	22.0	−10.8
Qwen3-1.7B	.65	34.4	−2.8	.42	26.8	−3.0
Qwen3-4B (hybrid)	.67	29.3	−4.1	.45	25.8	−4.4
Qwen3-8B	.68	31.4	−0.3	.47	26.1	−4.1
Phi-4-mini (3.8B)	.65	31.4	−10.2	.45	28.4	−0.7
Granite-3.3 (2.5B)	.62	26.6	−9.4	.39	25.8	−4.7
SmolLM2 (1.7B)	.59	31.4	−7.6	.33	24.8	−4.9
Qwen3-4B-2507 (ours)	.70	34.8	−	.50	28.8	−

Table 5: **Prefill size.** Each prefill feeds both readouts, refitted with the paper’s recipes on the training problems; Omni-MATH and MMLU-Pro test sets, billed costs. R^2 : test log-length R^2 averaged over routes. vs. median: cost saved relative to median pricing with the same row’s success predictions. vs. ours: cost saved by the row’s router relative to ours (negative: it spends more); its paired intervals (300 resamples) include zero for Qwen3-1.7B and 8B on Omni-MATH and Phi-4-mini on MMLU-Pro.

Is the prefill router’s success pipeline needed? The prefill router (Varshney et al., 2026) selects a layer, a pooling and a PCA dimension per target by cross-validation over the encoder’s upper layers, concatenates the selected features and trains an ensemble of shared-trunk MLPs. We reimplemented this pipeline on the same encoder, splits and cost readout, and compared it with our per-route logistic readouts on fixed layers. The two are tied on all three test sets: ours saves −0.1% [−7.7, 4.0] on LCB, 1.1% [−1.0, 3.0] on MMLU-Pro and −2.2% [−5.7, 1.9] on Omni-MATH relative to it (success AUC .857 vs. .839, .715 vs. .714 and .829 vs. .842). Linear readouts on a fixed set of layers are enough, and they make the cost readout a drop-in addition: the same features feed both.

New model families. Readouts for Qwen3-32B and GLM-4.7-flash, fitted on the same frozen 4B features, were evaluated on 1,905 MMLU-Pro test problems (log-length R^2 .48 and .06). With all seven routes, prefill cost readouts save 30.0% [25.4, 35.3] relative to median pricing, 21.8% [16.3, 26.7] relative to cost from success and 20.6% [15.3, 25.4] relative to difficulty bins. On a pool of the two gpt-oss-20b routes and the two new families, they save 32.2%, 11.6% and 14.8%. Readouts onboarded from ten examples per new route match the full readouts in the seven-route pool (0.1 points) and trail them by 11.9 [−4.2, 21.9] in the new-family pool.

10 Deployment

Routing versus cascades. A cascade calls routes in sequence and stops when an answer is accepted (Chen et al., 2023; Aggarwal et al., 2023). To bound what any cascade could achieve on our pools, we give it a perfect judge: every ordered plan of the five routes is called cheapest first, each attempt is judged by its true outcome at no cost, and the cascade stops at the first correct answer. A learned judge can only do worse. Even against this bound, one-shot routing with our readouts spends 29.7% [19.6, 37.7] less at matched accuracy on LCB and 31.9% [26.4, 37.3] less on Omni-MATH, because a cascade pays for every failed attempt before the one it accepts. On MMLU-Pro the two tie (−0.8% [−6.1, 3.9]): the cheapest route solves 62% of questions at about 6% of the most expensive route’s cost, so failed first attempts are cheap.

Per-query budgets. Predicted costs also enforce per-query budgets. Under a hard cap on predicted cost per query on LCB, the constant-price rule needs $1.00\text{--}1.89\times$ our budget to reach the same accuracy (geometric mean $1.27\times$).

Onboarding a new route. Adding a route requires its own readouts. Holding out one route on LCB and fitting its success and cost readouts on k problems, the router saves 30.0% ($k=5$), 30.6% ($k=10$) and 32.8% ($k=50$) relative to median pricing, against 36.9% with full readouts (list prices). The held-out route matters most when it is deepseek-v4-flash: without it the router spends 35.4% more than median pricing on the full pool, while onboarding it from five problems turns this into a 26.1% saving; on MMLU-Pro, the same goes from −19.6% to 26.9% ($k=5$) and 32.6% ($k=10$), against 42.7% with full readouts.

TODO: Deployable policies (calibration-chosen V ; pinned `deploy_matched.txt`) and provider endpoints in full (4.A.37–4.A.41, 4.A.55). Onboarding and budget numbers above are at list prices (`decompose.py`); rerun billed if kept.

11 Pre-registered predictions

TODO: Two-sided pre-registration record: screen rule (4.A.9), GAIN confirmed Omni, MMLU-Pro; APPS GAIN not confirmed unpinned (pinned 16.8% billed); AIME NO GAIN falsified (gain 14.9%) and why (single-draw cheapest-route screen understates predictability); remaining calls (K&K, SuperGPQA, BBEH) to run.

12 Discussion and Limitations

When does per-query cost pay? It pays when output length varies within a model more than prices differ between models: here lengths span $11\text{--}121\times$ within a route (Fig. 1) while per-call costs span $17\text{--}29\times$ across routes (Table 2), leaving 45–60% headroom over median pricing. It is captured when that variation is legible from the prompt. Where length tracks difficulty (LCB, Omni-MATH), even difficulty-only pricing captures most of it; where it tracks the work a question requires (MMLU-Pro), only a dedicated readout does, and that is also where we capture least (49%), leaving the most room for better cost predictors.

Limitations. Each route needs output-length labels. The main experiments use five routes from two model families; two further families (Qwen3-32B, GLM-4.7-flash) were tested on MMLU-Pro only. Omni-MATH and MMLU-Pro test problems have one draw per route. Intervals are pointwise and assume independent problems, and Table 1 conditions on the fitted readouts; resampling the training problems as well and refitting every estimator widens them, but our saving over median pricing stays positive on both large test sets (Omni-MATH [23.2, 38.0], MMLU-Pro [22.9, 30.7]). Only deepseek-v4-flash is pinned to a provider; the gpt-oss routes are not, since their provider changes price but barely length. The prefill comparison covers four model families at up to 8B. Billed rates were measured over one collection, and prices change within days. The ZeroRouter, MixLLM and CARROT baselines are reimplementations.

13 Conclusion

A frozen prefill that already predicts success also predicts cost. On held-out test problems at billed prices, its cost readouts reduce spending at matched accuracy and outperform every alternative estimator we tested; pricing from difficulty alone matches them only where output length tracks difficulty. Endpoints of one model share these readouts up to an offset, so the readouts can absorb provider drift from a few dozen calls, though pinning a provider is simpler.

References

- Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, et al. AutoMix: Automatically Mixing Language Models, 2023. arXiv:2310.12963.
- Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025.
- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance, 2023. arXiv:2305.05176.
- Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A Unified Approach to Routing and Cascading for LLMs, 2024. arXiv:2410.10347.
- Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A Graph-based Router for LLM Selections, 2024. arXiv:2410.03834.
- Bofei Gao, Feifan Song, Zhe Yang, et al. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models, 2024. URL <https://arxiv.org/abs/2410.07985>. arXiv:2410.07985.
- Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations*, 2025.
- Liang He, Jingbo Wen, Yixiong Chen, Yue Yang, Qizhen Lan, Kangning Cui, and Xilu Wang. You cannot pick a provider from the price list: Market-aware routing for open-weight LLM inference, 2026.
- Qitian Jason Hu, Jacob Bieker, Xiuyu Li, et al. RouterBench: A Benchmark for Multi-LLM Routing System, 2024. URL <https://arxiv.org/abs/2403.12031>. arXiv:2403.12031.
- Naman Jain, King Han, Alex Gu, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, 2024. URL <https://arxiv.org/abs/2403.07974>. arXiv:2403.07974.
- Mehran Kazemi, Bahare Fatemi, Hritik Bansal, et al. BIG-Bench extra hard. *arXiv preprint arXiv:2502.19187*, 2025.
- Haorui Li et al. When is the same model not the same service? a measurement study of hosted open-weight LLM APIs, 2026.
- Guanjie Lin, Yinxin Wan, Shichao Pei, Ting Xu, Kuai Xu, and Guoliang Xue. Behavioral consistency and transparency analysis on large language model API gateways, 2026.
- William Lugoloobi, Thomas Foster, William Bankes, and Chris Russell. LLMs Encode Their Failures: Predicting Success from Pre-Generation Activations, 2026. arXiv:2602.09924.
- M-A-P Team. SuperGPQA: Scaling LLM evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.
- Glen Messenger. ALPS: Activation-Based Length Prediction for Intelligent LLM Inference Scheduling, 2026. URL <https://doi.org/10.5281/zenodo.19078431>. Zenodo. doi:10.5281/zenodo.19078431.

- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, et al. RouteLLM: Learning to Route LLMs with Preference Data, 2024. arXiv:2406.18665.
- Zhihong Pan, Kai Zhang, Yuze Zhao, and Yupeng Han. Route to Reason: Adaptive Routing for LLM and Reasoning Strategy Selection, 2025. arXiv:2505.19435.
- Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, et al. CARROT: A Cost Aware Rate Optimal Router, 2025. URL <https://arxiv.org/abs/2502.03261>. arXiv:2502.03261.
- Seongho Son, Sangwoong Yoon, Jiahua Tang, Shuhan Wang, et al. SWE-Router: Routing in Multi-turn Agentic Software Engineering Tasks, 2026. arXiv:2607.00053.
- Wei Song, Zhenya Huang, Cheng Cheng, Weibo Gao, et al. IRT-Router: Effective and Interpretable Multi-LLM Routing via Item Response Theory, 2025. arXiv:2506.01048.
- Tanay Varshney, Annie Surla, Michelle Xu, et al. LLM Router: Rethinking Routing with Prefill Activations, 2026. URL <https://arxiv.org/abs/2603.20895>. arXiv:2603.20895.
- Xinyuan Wang, Yanchi Liu, Wei Cheng, et al. MixLLM: Dynamic Routing in Mixed Large Language Models, 2025. URL <https://arxiv.org/abs/2502.18482>. arXiv:2502.18482.
- Yubo Wang, Xueguang Ma, Ge Zhang, et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark, 2024. URL <https://arxiv.org/abs/2406.01574>. arXiv:2406.01574.
- Huanyi Xie, Yubin Chen, Liangyu Wang, et al. Predicting LLM Output Length via Entropy-Guided Representations, 2026. URL <https://arxiv.org/abs/2602.11812>. arXiv:2602.11812.
- Cheng Yan, Wuyang Zhang, Zhiyuan Ning, et al. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space, 2026. URL <https://arxiv.org/abs/2601.06220>. arXiv:2601.06220.
- John Yang, Kilian Lieret, Carlos E. Jimenez, et al. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.

A Pool statistics

TODO: Per pool: n, splits, draws per route, accuracy ladder, output p10/p50/p90, billed prices (pinned).

B Reimplemented baselines

TODO: ZeroRouter (choices, D/K sensitivity, own encoder, population curve; 4.A.30–4.A.36), MixLLM/CARROT-style, prompt GBM (cite latencyrouting).

C Protocol details

TODO: Frontiers/hulls, matched-accuracy band, paired and refit bootstraps, price fitting and cross-fitting, provider pinning.